

Monthly Report Calculation Specification

Reference for app implementation. Specifies exactly how each metric is computed from the four source files.

Source Files

File	Description
Leads.csv	Standard CSV with header row. GymSales lead export. All leads ever created (not filtered by month).
Sales.csv	CSV with 2 header lines before data. Skip first 2 rows; row 3 is the column header. ABC POS agreements.
Member_Snapshot.csv	CSV with 2 header lines before data. Skip first 2 rows. ABC POS active members at time of pull.
RFC.csv	Standard CSV with header row. Members removed for collections this month.
Cancel list	Pasted text. One row per line. Format varies. Identify revocations by reason text.

Pre-Processing

1. Filter Sales to real agreements

The Sales file contains many empty grouping rows. Keep only rows where **Agreement #** is a numeric string.

```
sales = sales[sales['Agreement #'].str.match(r'^\d+$')]
```

2. Parse dates

Leads **created_at** and **waiver_signed_date** are timezone-aware strings (e.g., 2026-04-15 14:30:00 -0400). Parse as UTC, then convert to US/Eastern. Sales **Queue Date** is a plain date string with no timezone. RFC **Status Date** and **Begin Date** are also plain date strings.

3. Normalize plan names

Plan names in Sales contain double spaces. Collapse whitespace before any comparison:

```
plan_clean = re.sub(r'\s+', ' ', plan_raw).strip()
```

4. Plan exclusions (apply to all sales/member counts)

Exclude these three short-term student plans from every sales and membership count:

```
EXCLUDED_PLANS = {
    'Student 1 Month PIF',
    'Student Basic Power 2 Mo PIF - Floor Plan',
    'Student Basic Power 3 Mo PIF - Floor Plan',
}
```

Include all other plans: 12-Month, Month-to-Month, PIF in-club, Trainer plans, and any student plans on 12-month contracts.

5. Name normalization for matching

Lead names and sale names need shared normalization to match across files. Handles apostrophes (O'Leary), hyphens (Meier-Viereck), and middle initials (Jones, Dondre D).

```
def normalize(text):
    text = text.lower().strip()
    text = text.replace("'", "").replace("-", " ")
    text = re.sub(r'\s+', ' ', text)
```

```

    return text

# Lead names: combine first + last, drop middle words from first
def norm_lead(first, last):
    first_clean = first.lower().strip().split()[0] if first.strip() else ''
    return f"{normalize(first_clean)} {normalize(last)}.strip()"

# Sale names (ABC format "Last, First M")
def norm_sale(name):
    parts = name.split(',', 1)
    if len(parts) != 2:
        return normalize(name)
    last = parts[0].strip()
    first_raw = parts[1].strip()
    first = first_raw.split()[0] if first_raw else ''
    return f"{normalize(first)} {normalize(last)}.strip()"

```

6. Time period

Calendar month, Eastern time. April 2026 = 2026-04-01 00:00:00 ET through 2026-05-01 00:00:00 ET (exclusive end).

Lead Channel Attribution

Channel is derived once at lead creation from **source**, **status**, and **tags**. Channel never changes after creation.

```

def channel(lead):
    source = lead['source']
    status = lead['status']
    tags = (lead['tags'] or '').lower()

    if source == 'Website':
        return 'Web'
    if 'online-join' in tags or 'online join' in tags:
        return 'Web'
    if source == 'Visitor Registration App':
        return 'Guest' if status == 'guest' else 'Walk-in'
    return 'Walk-in' # Walk-In, Member Referral, blank, etc.

```

Locked rules:

- Source field is set at lead creation and never changes. Channel never changes.
- A Web lead who later signs the iPad waiver remains a Web lead. The waiver event is a milestone, not a new lead.
- Guest leads are excluded from all lead counts and conversion math.
- The online-join tag exception covers ~5% of online-join records where ABC writes back without populating the source field.

Sale Channel Attribution

For each sale closed in the report month, match it to a lead by normalized name. Pick the most recent prior lead. Prefer non-Guest matches when available.

```

def attribute_sale(sale):
    name = norm_sale(sale['Member Name'])
    sale_date = sale['Queue Date']

    candidates = leads[leads['norm'] == name]
    if len(candidates) == 0:
        return ('Walk-in (no lead)', None)

    valid = candidates[candidates['created_at_et'].date() <= sale_date.date()]
    if len(valid) == 0:

```

```

    valid = candidates

    valid = valid.sort_values('created_at_et', descending=True)

    non_guest = valid[valid['channel'] != 'Guest']
    best = non_guest.iloc[0] if len(non_guest) > 0 else valid.iloc[0]

    channel = best['channel']
    if channel == 'Guest':
        channel = 'Walk-in'

    return (channel, best['created_at_et'])

```

Section 1: Lead Generation

Metric	Calculation
Web Leads	Count of leads where created_at_et is in the report month AND channel == 'Web'.
Walk-in Leads	Count of leads where created_at_et is in the report month AND channel == 'Walk-in'.
Total Leads	Web Leads + Walk-in Leads. Guest leads are excluded.

Section 2: Sales

Apply plan exclusions first. Then for each sale where Queue Date is in the report month:

Metric	Calculation
Web Sales	Count where attributed channel == Web.
Walk-in Sales	Count where attributed channel == Walk-in.
Total Sales	Web Sales + Walk-in Sales. No Direct category — every sale is one or the other.

Section 3: Conversion

Web Visit Conversion

Percent of Web leads created in the month who signed the iPad waiver in the same month.

```

visited = web_leads_in_month[
    waiver_signed_at_et >= month_start AND
    waiver_signed_at_et < month_end
]
web_visit_conversion = len(visited) / len(web_leads_in_month)

```

Web Visit-to-Sale Conversion

Of the Web leads who visited (signed waiver) in the month, what percent bought in the month.

```

visited_and_bought = visited[norm in sales_month_norm_set]
visit_to_sale = len(visited_and_bought) / len(visited)

```

Web Sales Conversion

Percent of Web leads created in the month who closed (a sale) in the month, regardless of whether they visited.

```

web_sales_conversion = web_sales_in_month / web_leads_in_month

```

Note: Numerator counts sales matched to any prior Web lead, including Web leads from earlier months that converted in the report month. Denominator is leads created in the report month. This follows the locked methodology, even though it mixes vintages slightly.

Walk-in Sales Conversion

```
walkin_sales_conversion = walkin_sales_in_month / walkin_leads_in_month
```

Section 4: Losses

Metric	Calculation
Cancels	From the pasted cancel list. One per line. Exclude any line where the reason indicates gym-initiated termination
RFC	Row count of RFC.csv.
Revocations	From the cancel list, count of lines with gym-initiated termination language.
Pending Cancel	Count of members in Member_Snapshot.csv where Member Status == 'Pending Cancel' (after applying plan ex

Section 5: Membership

Plan-Eligible Member Base

From Member_Snapshot.csv, filter to rows with numeric Agreement # and Plan_clean not in EXCLUDED_PLANS.

Active Member Base (Current Member Base)

Plan-eligible members minus those with Member Status == 'Pending Cancel'.

```
current_active = plan_eligible - pending_cancel
```

Start-of-Month Member Base

Reverse-calculated from the snapshot. Pending Cancel is not included in losses for this calculation - they are still active for start-of-month purposes.

```
start_of_month = current_member_base - new_sales_in_month + losses_in_month  
# losses = cancels + RFC + revocations
```

Net Gain

```
net_gain = total_sales - (cancels + RFC + revocations)
```

Attrition Rate

```
attrition_rate = (cancels + RFC + revocations) / start_of_month_base
```

Pending Cancel is reported separately as a forward-looking indicator and is not in the attrition numerator.

Section 6: Pipeline Velocity (Cumulative)

For each sale closed in the month with a matched lead, calculate days from lead creation to sale:

```
days_to_close = (sale_date.date() - lead_created_at.date()).days
```

Bucket each sale (using calendar days, sale date inclusive):

Bucket	Range
Same day	days_to_close <= 0
1-7	1 <= days_to_close <= 7
8-30	8 <= days_to_close <= 30

Bucket	Range
31+	days_to_close >= 31

Report cumulative percentages per channel - each column counts sales closed within that timeframe or sooner:

```

within_same_day = count(bucket = 'Same day')
within_7        = within_same_day + count(bucket = '1-7')
within_30       = within_7        + count(bucket = '8-30')
within_31_plus  = within_30       + count(bucket = '31+')    # equals total

```

Each percentage is $(\text{cumulative_count} / \text{total_channel_sales}) * 100$. Direct sales without a lead match should be near zero with proper fuzzy matching. If any remain, treat them as same-day for velocity purposes.

Promo Cohort Tracking

Used for past-due cohort analysis and retention tracking. Not in the headline report. A sale's cohort is determined by Queue Date.

Cohort	Sale Date Range
Pre-sale	Nov 1, 2025 - Nov 7, 2025
NYE/Jan promo	Dec 29, 2025 - Jan 6, 2026
Valentine promo	Feb 13, 2026 - Feb 15, 2026
End-Feb promo	Feb 27, 2026 - Feb 28, 2026
Easter promo	Apr 3, 2026 - Apr 6, 2026
May promo	May 2, 2026 - May 5, 2026
Organic	All other dates

New promo windows must be added explicitly each time a promo runs.

Past-Due / RFC Forecast Workflow

Daily delinquency report from ABC contains: Member Name, Phone, Email, Plan, Days Past Due, Total Past Due. The app should:

1. Match each past-due name to its original sale by normalized name.
2. Pull Queue Date from the original sale to assign cohort.
3. Calculate RFC Estimate Date = today + (45 - days_past_due). 45-day RFC trigger.
4. Flag urgency tier (see below).

Days Past Due	Tier Action
1-7 days late	Auto-retry, no action
8-14 days late	Text + call
15-25 days late	Manager call + ABC flag
26-35 days late	Final notice all channels
36-44 days late	Documented final outreach
45+ days late	RFC imminent, stop active outreach

Sort output: promo cohorts first (most recent promo on top), then pre-sale, then organic. Within cohort, sort by RFC date ascending (most urgent first).

Validation Checks Before Publishing

- **Sales reconcile:** Web Sales + Walk-in Sales = Total Sales
- **Velocity rows reconcile:** each channel's Within 31+ days count equals its Total Sales
- **Member math reconciles:** Start + New Sales - Losses = Current
- **Conversion ratios within sanity bounds:** conversion rates should fall between 5% and 40%; flag if outside
- **No zero-denominator divisions:** handle months with zero leads in a channel gracefully
- **Promo windows up to date:** if any sale falls in a known promo date but isn't tagged, alert

Known Edge Cases the App Must Handle

1. **Sales reporting through end of next month.** ABC's "Sales by SIGN Date" report often extends 1-2 days past the report month. Filter strictly to month boundaries.
2. **Both EST and EDT offsets in the lead file.** The DST transition means created_at strings can have either -0400 or -0500. Parse with timezone awareness, then convert to Eastern.
3. **Ambiguous name matches.** Multiple leads may share a normalized name (e.g., several Mark Jones). Resolve by picking the most recent lead created on or before the sale date.
4. **Sales appearing before a matching lead.** Rare but happens. Fall back to using the closest-after-sale lead if no prior match exists.
5. **Plan name whitespace.** ABC writes some plans with double spaces. Always collapse whitespace before comparing to exclusion list.
6. **Cancel list format drift.** The cancel list comes pasted, format varies. Expect at minimum a date, a name, optional reason text. Detect revocations from reason language, not from a structured field.